

פרק 1,2 - מבוא

- אלגוריתם - הליך חישובי מוגדר היטב המקבל קלט ומוציא פלט. בעיה קלאסית - בעיית המיון. אלגוריתם הוא נכון אם לכל מופע הוא מסתיים בפלט נכון.
- לפעמים יש באלגוריתמים לא נכונים תועלת אם הטעות שלהם היא ידועה.
- הערך באלגוריתמים - חוסכים משאבים
- אם למחשב היה אינסוף זיכרון והמעבד היה מהיר אינסופית לא היה ערך באלגוריתמים אבל זה לא המצב.
- אלגוריתמים הם טכנולוגיה - מאפשרים יכולות רבות, ממש כמו לקנות עוד חומרה.
- בעיית הסוכן הנוסע - לא ידוע על אלגוריתם שפותר אותה אבל יש שיטות למצוא מרחק קטן שלא רחוק מהקטן ביותר האפשרי.
- הוכחת נכונות באמצעות שמורת הלולאה - נכונות כללים מסוימים בעת אתחול, תחזוקה, סיום.
- פסאודו קוד: $x \leftarrow value, if - then - else, for j \leftarrow 2 to length[A], while, repeat$
- מקרים - טוב, ממוצע, גרוע. הגרוע מעניין מאוד כי זה מלמד על כל המקרים.
- באלגוריתם אקראי נבדוק את התוחלת
- אנו משווים את שיעור/קצב הגידול של זמני הריצה - זה הנתון העיקרי המעניין.
- ההערכה עלולה להיות שגויה עבור קלטים קטנים.
- גישות - אינקרמנטלי (מעבדים איבר-איבר), הפרד ומשול (מחלקים את הבעיה לכמה תתי בעיות, לרוב ברקורסיה).
- עץ רקורסיה
- כל קריאה היא ענף, כל צומת היא עלות חישוב, הגובה של העץ הוא מספר הקריאות, לא תמיד העץ מאוזן, הסכום הכולל הוא היעילות, לרוב הוא סכום של רמה כפול סכום באותה הרמה.
- 2.3-5: חיפוש בינארי - חיפוש במערך ממויין על ידי הקטנת הטווח פי 2 בכל פעם.
- אלגוריתם למציאת מספרים שסכומם S - מיון המערך ואתחול שני אינדקסים בקצוות, קידום כל קצה בהתאם להאם הסכום גדול או קטן מדי.
- בעיה 2-1: מיון הכנסה על מערכים קטנים במיון מיזוג - ניתן לשפר את זמן הריצה של מיון מיזוג על ידי ביצוע מיון הכנסה על מערכים קטנים בגודל קבוע, זמן הריצה יהיה $\Theta\left(nk + n \lg \frac{n}{k}\right)$
- בעיה 2-2: מיון בועות - n פעמים עוברים על המערך עד להתחלה ומזיזים את המספר הכי קטן הכי שמאלה. בתחילת המערך יש את המערך הממויין שגדל עם הזמן.
- בעיה 2-3: כלל הורנר - חישוב פולינום בזמן $\Theta(n)$ במקום מימוש קלאסי של $\Theta(n^2)$. $p(x) = ((a_n)x + a_{n-1})x + \dots$ יהי
- בעיה 2-4: היפוך הוא כאשר מספר קטן מופיע אחרי מספר גדול, קיימת דרך לחשב את כל ההיפוכים באמצעות שינוי קטן ל $mergesort$.
- מיון הכנסה יציב - כי המספרים מוכנסים אחד אחד מהאזור שאחרי המערך, ומחפשים את המקום המתאים מהסוף להתחלה
- מיון מיזוג יציב - כי החלק השמאלי קודם לימני (יש בקוד $\geq$).

פרק 3 - גידול של פונקציות

• סימונים - $O, o, \Omega, \omega, \Theta$ (בסיכום החיצוני)

• הערות

- פונקציות צריכות להיות אי-שליליות עבור n גדול.
- אפשר לכלול סימונים בתוך משוואה אם אין לנו עניין להיכנס לפרטים - לדוגמה $n^2 + \frac{1}{2}n + \frac{1}{2} = n^2 + \Theta(n)$.
- מספר הפונקציות האלמוניות בביטוי הוא מספר המופעים בביטוי - כלומר $\sum O(n)$ הוא פונקציה אחת (עדיין לא ממש הגיוני אבל נעזוב)
- אם מופיעות פונקציות בשני צידי המשוואה (לדוגמה $2n^2 + \Theta(n) = \Theta(n)$) סימן שקיימים קבועים המקיימים את המשוואה.

• יחסים

- טרנזיטיביות - כולם
- רפלקסיביות - O, ω, Θ
- סימטריות - Θ , סימטריות מוחלפת - $(O, \Omega), (o, \omega)$.
- Θ אם ורק אם O ו- Ω
- קיימת הקבלה עם הסימונים $\leq, \geq, <, >$
- * אמנם לא ניתן להשוות אסימפטוטית שתי פונקציות (לדוגמה פונקציות מחזוריות ינועו סביב סט ערכים שוב ושוב).

$$\text{דוגמה מוגדרת: } g(n) = \begin{cases} n & n = 2p \\ \frac{1}{n} & n = 2p + 1 \end{cases} \text{ עם כמעט כל פונקציה אחרת.}$$

• סימונים מתמטיים

- פולינום חיובי אסימפטוטית המקדם העליון a_d חיובי.
- פונקציה חסומה פולינומית אם ורק אם קיים k כך ש- $f(n) = O(n^k)$. חסומה פולינומית אם ורק אם קיים k כך ש- $f(n) = O(\lg^k n)$
- לוגריתם חוזר $\lg^* n$ כמה פעמים צריך לעשות $\lg$ רקורסיבית עד שמגיעים ל-1.
- בעיה 3-1: אם $p(n)$ פולינום כלשהו ממעלה a אז היחס בין $p(n), n^b$ נקבע לפי היחסים בין a, b .
- בעיה 3-2:

$$\text{- אם } f = O(g) \text{ אז לא בהכרח } 2^f = O(2^g) \text{ - דוגמה } n, 2n.$$

$$\Omega(f) + o(f) = \Omega(f) -$$

$$\text{• כל פונקציה פולינומית חיובית גדלה יותר מהר מכל פונקציה לוגריתמית } \lg^b n = O(n^a).$$

• הערות

- כדי להראות o/ω - אפשר להראות שהגבול שואף ל- $0/\infty$. אמ"מ - אם לא אז זה לא. אפשר להוכיח על ידי הכפלה ושימוש בהגדרה.
- אינטואיציה - O גדול אומר שקיים קבוע נכפיל בו ונגדיל מספיק את n אז אחת תהיה גדולה מהשניה. o קטן אומר שלא משנה איזה קבוע זה יהיה אז התנאי יתקיים.

• סדרי גודל של פונקציות:

$$\left(1, n^{\frac{1}{\lg n}}\right), \lg \lg^* n, (\lg^* n, \lg^* \lg n), 2^{\lg^* n}, \ln \ln n, \sqrt{\lg n}, \ln n, \lg^2 n, 2^{\sqrt{2 \lg n}}, \sqrt{2^{\lg n}}, (2^{\lg n}, n), (n \lg n, \lg(n!)), (n^2, 4^{\lg n}) \dots$$

$$\dots (n^{\lg \lg n}, \lg n^{\lg n}), (\lg n)!, \left(\frac{3}{2}\right)^n, 2^n, n2^n, e^n, n!, (n+1)!, 2^{2^n}, 2^{2^{n+1}}$$

$$(\lg n^{\lg a}, a^{\lg \lg n})$$

$$(c^{\lg n}, n^{\lg c}) \quad (\text{prove by } n = 2^{\lg n}, c = 2^{\lg c})$$

פרק 4 - נוסחאות נסיגה

• פרטים טכניים

- מוגדר רק עבור מספרים שלמים אז צריך לציין גם ערכי רצפה/תקרה. לפעמים הם כן מהותיים ויש להתייחס אליהם.
- שינוי $T(1)$ יגרום לשינוי בקבוע בסה"כ, לכן לא מהותי להתייחס אליו. בכללי עבור ערכי n קטנים זמן הריצה הוא $\Theta(1)$.

• שיטת ההצבה

- שלבים: ניחוש הפתרון הכללי, הוכחה באמצעות אינדוקציה (מוכיחים ל-1, מניחים ל- $n-1$ ומוכיחים ל- n).
- אפשר להתחיל עם חסם עליון ותחתון חלשים ולהתקדם.
- אי דיוקים - לפעמים מקבלים תוצאה קטנה/גדולה במספר מסוים, אם כך אפשר להוסיף/לחסר מהניחוש המקורי.
- * לדוגמה עבור $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$ (עמוד 56) והניחוש cn מקבלים $T(n) \leq cn + 1$. הניחוש $cn - b$ עובד, כאשר b נקבע כרצוננו.
- לשים לב להוכיח בדיוק את האינדוקציה.
- * לדוגמה אם $T(n) = T(\lfloor \frac{n}{2} \rfloor) + n$ אז צריך להוכיח $T(n) \leq cn$ ולא $T(n) \leq cn + n$.
- החלפת משתנים - ניתן להחליף משתנים וגם את נוסחת הנסיגה בביטוי שקול.
- * לדוגמה $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$ אפשר לסמן $n = 2^m, S(m) = T(2^m)$. נקבל: $T(n) = T(2^m) = S(m) = 2S(\lfloor \frac{m}{2} \rfloor) + m = O(m \lg m) = O(\lg n \lg \lg n)$.

• שיטת עץ הרקורסיה

- כל קריאה היא ענף, כל צומת היא עלות חישוב, הגובה של העץ הוא מספר הקריאות, לא תמיד העץ מאוזן, הסכום הכולל הוא היעילות, לרוב הוא סכום של רמה כפול סכום באותה הרמה.

- לדוגמה: $T(n) = 3T(\frac{n}{4}) + cn^2$, אז

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) = \frac{\left(\frac{3}{16}\right)^{\log_4 n} - 1}{\frac{3}{16} - 1} + \Theta(n^{\log_4 3}) \stackrel{\text{חסם עליון}}{<} \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i + \Theta(n^{\log_4 3}) = \frac{13}{16}cn^2 + \Theta(n^{\log_4 3}) = O(n^2)$$

• לא תמיד העץ הוא עץ מלא, אבל עדיין אפשר להשתמש בשיטת ההצבה כדי לחשב חסם.

פרק 5 - ניתוח הסתברותי ואלגוריתמים אקראיים

• אלגוריתמים אקראיים

- מאפשרים להבטיח תוחלת ללא תלות בקלט, כמובן על חשבון הגדלת הזמן של המקרה הטוב ביותר.
- תמורות אקראיות במערכים - איך אפשר לעשות שאפל למערך?
 - * לתת לכל מספר עדיפות ואז למיין לפי העדיפויות.
 - * לעבור על המערך ולהחליף כל איבר עם איבר רנדומלי כלשהו

פרק 6 - מיון ערימה

• עצים (הורה, שמאל, ימין), תכונת הערמה (מקסימום / מינימום)

– $max - heapify$ - מורידה את הצומת הנתון כמה שיותר נמוך בעץ (רקורסיבית, עוצרת כשאי אפשר)

– $build - max - heap$ - עוברת על כל האבות בעץ בסדר יורד ומורידה את הצומת כמה שיותר נמוך. בסדר יורד כי אחרת ייתכן שצומת לא ירד עד הסוף, לדוגמה $[2, 1, 1, 3]$.

– $heapsort$ - בונה ערמה, כלולאה מחליפה את השורש עם האיבר האחרון ואז עושה $max - heapify$

* מיון מערך בסדר עולה או יורד עדיין $n \lg n$

* מיון ערימה לא יציב - מיון ערימת מינימום עם קלט 221 (הראשון יעקוף את ה-2 השני)

– $heap - delete$ - החלפת ערך האיבר למחיקה ב- ∞ , פעפוע לשורש, החלפת השורש עם סוף הערימה והקטנתה, $max - heapify$ על הראש.

• תכונות

– מספר איברים מינימלי - 2^h , מקסימלי - $2^{h+1} - 1$, בעומק h יש בין 1 ל- 2^h צמתים, גובה ערמה בת n איברים הוא $\lfloor \lg n \rfloor$, העלים הם $1 \dots n + \lfloor \frac{n}{2} \rfloor$

– יש עד $\lfloor \frac{n}{2^{h+1}} \rfloor$ צמתים בגובה h !

– הערך הגדול ביותר הוא בשורש

– ערימה אינה בהכרח מערך ממויין

• תור קדימויות

– שגרות:

* העלאת ערך של צומת ($heap - increase - key$) - עדכון המפתח של איבר מסוים אם המפתח גדול מערך האיבר, ואז פעפוע למעלה של הערך עד לקבלת ערמה תקינה.

* הכנסת ערך - הגדלת גודל הערמה, הכנסת $-\infty$ בסוף ואז העלאת ערך הצומת לפי הערך הנתון.

* לבחון / להוציא את המקסימום - החזרת השורש וזהו או החלפה עם הסוף וביצוע $max - heapify$

– שימוש - מערכת שמתווספות אליה משימות וכל פעם לוקחת את המשימה עם הקדימות הגבוהה ביותר.

– מיזוג k רשימות ממויינות כאשר n מספר כולל בזמן $O(n \lg k)$ - בניית ערימת מינימום מכל הראשים של האיברים, כאשר איבר יוצא החוצה מכניסים את האיבר הבא מאותה הרשימה.

פרק 7 - מיון מהיר

• שגרות

– *partition* - מגדירה את המספר האחרון בתור איבר הציר, מגדירה החל מההתחלה מערך של מספרים קטנים ואז גדולים, וסורקת את המערך ומוסיפה כל פעם למערך המתאים. אם מספר מסוים קטן אז הוא מוחלף עם המספר הגדול הכי קטן. בסוף מחליפים את איבר הציר עם האיבר הגדול הכי קטן.

* אם כל האיברים שווים זה לזה - כל האיברים נכללים בתור המערך של הקטנים

– *quicksort* - חלוקת כלל המערך, מיון של מערך ה"קטנים" ומערך ה"גדולים"

* מקרה טוב - איבר הציר הוא החציון (למעשה כל חלוקה ביחס קבוע). מקרה גרוע - כל חלוקה היא $0, n-1$.

* צירוף של חלוקה טובה ואז חלוקה גרועה יוצר חלוקה טובה

* מערך של איברים זהים זהה למקרה הגרוע.

* מיון מהיר לא יציב - 3312 (ה-3 ימים יחליפו ביניהם).

– *randomized - partition, randomized - quicksort* - איבר הציר נבחר באופן אקראי

• הערות

– עבור מערך כמעט ממויין עדיף מיון-הכנסה שרץ ב- $\Theta(cn)$ במקום מיון-מהיר (החלוקה תהיה במקרה הטוב $c, n-c$ וזה מוביל ל- n^2) אפשר לעשות אלגוריתם שממייין עם מיון מהיר עד אורך k ואז ממייין כל רשימה עם מיון הכנסה, ואז מיון הכנסה על המערך כולו. במצב זה זמן הריצה הוא $O\left(nk + n \lg \frac{n}{k}\right)$.

פרק 8 - מיון בזמן לינארי

• עץ החלטה

- עץ החלטה - כל צומת היא השוואה, כל קשת היא תוצאה של ההשוואה, כל עלה הוא סידור אפשרי של המערך.
- 8.1 - כל אלגוריתם עושה $\Omega(n \lg n)$ השוואות במקרה הגרוע.
- * מיון-ערמה ומיון-מיזוג הם אופטימליים
- * הוכחה מתבססת על $n! \leq 2^h$
- עומק מינימלי אפשרי בעץ החלטה - $n - 1$ (מערך ממויין, נכון כי גרף ההחלטות צריך להיות מקושר).
- 8.1-2 - אפשר לחסום סכום על ידי חסימה בין שני אינטגרלים.
- 8.1-3 - אי אפשר לעשות מיון לינארי עבור חצי $\frac{1}{n}$ וכו' מהקלטים כי זה אומר שכל הפתרונות צריכים להיות בעומק מסוים אבל אין מספיק עלים - $n!$ גדלה מאוד מהר.

• מיון מניה

- אלגוריתם - סופרים כמה פעמים כל איבר נמצא, סוכמים את המערך כדי לקבל כמה מספרים קטנים או שווים למספר מסוים, ואז עוברים על האיברים במערך המקורי בסדר הפוך ולפי הערך במערך המונים ממקמים אותם במערך החדש.
- לוקח $\Theta(n + k)$
- יציב!
- לא מבוסס השוואות, מניח שהקלט הוא בטווח ערכים מסוים וגם משתמש במערך עזר שיכול לתפוס המון מקום לא לצורך.

• מיון בסיס

- אלגוריתם: למיין רשימה של n מספרים בעלי d ספרות ו- k ערכים אפשריים באמצעות מיון כל ספרה בנפרד מהנמוכה לגבוהה.
- * מתחילים מהנמוכה כי ככה לא צריך לשמור על "ערימות" נפרדות בתהליכי הביניים
- רץ בזמן $\Theta(d(n + k))$ (אם המיון היציב רץ בזמן $\Theta(n + k)$)
- * למה 8.4 - כאשר יש n מספרים בעלי b ספרות ושלם $r \leq b$, מיון-בסיס ממיין בזמן $\Theta(\frac{b}{r}(n + 2^r))$, $d = \frac{b}{r}$, $k = 2^r - 1$, כל ספרה היא בת r סיביות, ממיינים את הספרות לפי מיון מניה ועושים את זה d פעמים
- האם עדיף על מיון מהיר?
- * אם $b = O(\lg n)$ ו- $r \approx \lg n$ אז לכאורה הזמן הוא לינארי
- * האלגוריתם לא ממיין במקום
- * כל מעבר של מיון-בסיס לוקח יותר זמן מאשר מיון-מהיר.
- יציבות

* מיון-הכנסה ומיון-מיזוג יציבים, מיון-ערמה ומיון-מהיר לא.

* כדי להפוך ליציבים - להפוך כל איבר $tuple$ שמכיל גם את האינדקס ואז השוואה תשווה גם את האינדקס.

- 8.3-4: אפשר למיין n מספרים שלמים בתחום $(0, n^2 - 1)$ ב- $O(n)$ (ממירים לבסיס n (3 ספרות לפי הגדרה של בסיסי מספרים) ואז מיון-בסיס של 3 ספרות).

• מיון-דלי

- לחלק איברים לקבוצות לפי טווח ערכים ואז למיין כל קבוצה בנפרד.

פרק 9 - חציונים וערכי מיקום

- למצוא מינימום או מקסימום לוקח $n - 1$ השוואות וזו גם הדרך הטובה ביותר (כל איבר חייב להשתתף)
- למצוא מינימום ומקסימום ביחד לוקח $3 \lfloor \frac{n}{2} \rfloor$ השוואות - משווים כל זוג איברים ביניהם ואז למינימום ומקסימום נוכחיים.
- 9.1-1 ניתן למצוא את האיבר השני הכי קטן ב $n + \lceil \lg n \rceil - 2$ אם אורך המערך הוא 2 אז לוקחים את הגדול יותר. אם לא אז מחלקים את המערך לזוגות ומשווים אותם, ומבצעים עבור כל המספרים הקטנים רקורסיבית.
- 9.3-3 - ניתן לשפר את *quicksort* לזמן גרוע $O(n \lg n)$ אם כל האיברים שונים על ידי כך שקודם מוצאים את החציון ועושים חלוקה סביבו.
- בעיית הבחירה הכללית
 - בחירה אקראית
 - * רצה בתוחלת $\Theta(n)$. במקרה הגרוע היא $\Theta(n^2)$ אבל בגלל האקראיות אין באמת קלט גרוע.
 - * אלגוריתם - מבצעים *partitioning* לפי איבר רנדומלי במערך, מזה אפשר לדעת את ערך המיקום שלו, ולפי כך עושים *partition* על אחד מתת המערכים שנוצרו.
 - בחירה
 - * רצה תמיד ב- $O(n)$
 - * אלגוריתם - מחפשים את החציון על ידי חלוקה לתת מערכים, מציאת החציון שלהם ואז השוואת החציונים. ביצוע חלוקה לפי החציון.

פרק 10 - מבני נתונים בסיסיים

- דוגמאות לפעולות - חיפוש, הכנסה, מחיקה, מינימום, מקסימום, עוקב, קודם

- מחסנית - שליפה ממחסנית ריקה זה *underflow*, הכנסה למחסנית מלאה זה *overflow*

- תור

- מימוש במערך סטטי - בהכנסה מקדמים את הראש וכשמגיעים לסוף עושים לופ להתחלה. כאשר מוציאים אז מקדמים את הזנב.

- רשימה מקושרת

- זקפים - עצם דמה שמאפשר להתעלם ממקרי גבול

- ייצוג עצמים ללא מצביעים

- ניתן לייצג כמות של עצמים זהפים על ידי מערך לכל תכונה.

- * הקצאה ושחרור - ניתן לשמור רשימה מקושרת של הזיכרון הפנוי ובעת ניסיון להקצות זיכרון נוסף לקדם את הרשימה או להוסיף לרשימה.

- ניתן לייצג במערך אחד כמות של עצמים שונים על ידי שמירת ה"כתובת" (אינדקס) במערך ומשם חישוב היסט מתאים בהתאם לתכונה.

- הקצאה ושחרור

- עצים

- יש שיטה לשמירת מספר משתנה של בנים בעץ - כל צומת מכיל מצביע להורה, לבן שמאלי ולאח ימני

- כמות העלים בעץ בגובה h היא במקסימום 2^h . כל שכבה בגובה h מתחילה ב- 2^{h-1} ונגמרת ב- $2^h - 1$.

פרק 11 - טבלאות גיבוב

- טבלאות מיעון ישיר - כל מפתח מוביל לערך בודד

- טבלאות גיבוב

- פונקציית גיבוב ממפה מפתח לערך - מקטינה את טווח האינדקסים שצריך לטפל בהם.

- פתרון התנגשויות על ידי שרשור - יוצרים רשימה מקושרת מכל הערכים באותו תא.

- פונקציות גיבוב

- גיבוב עם מיעון פתוח

- שומרים כל איבר בטבלה עצמה, פונקציית הגיבוב מקבלת גם מספר ניסיון ובכך פולטת כל מפתח בטבלה.

פרק 12 - עצי חיפוש בינאריים

- תכונת עץ החיפוש - בן שמאלי קטן ממנו ובן ימני גדול ממנו
- סריקות - תחילית, תוכית, סופית.
- אפשר למיין ב- $\Theta(n)$ בסריקה תוכית.
- ערימה אי אפשר - לוקח $O(n \lg n)$
- יש אלגוריתמים לא רקורסיביים לסריקה תוכית (פתרון לתרגיל 10.4-6)
- עם מחסנית - כל עוד מחסנית לא ריקה ומצביע הוא לא כלום (כל עוד מצביע לא כלום (לך שמאלה עד הסוף ותדחוף בנים למחסנית ותעדכן מצביע), עכשיו תדפיס ואז לך אחד ימינה וחוזר חלילה.
- בניית עץ חיפוש לוקחת לפחות $n \lg n$ כי מיון n איברים לוקח לפחות $n \lg n$
- פעולות
 - חיפוש - החלטה ימינה/שמאלה עד שמוצאים
 - מינימום - הכי שמאלי, מקסימום - הכי ימני
 - עוקב - אם יש בן ימני אז העוקב הוא המינימום של תת העץ הימני. אם לא אז בלולאה מחפשים את ההורה הראשון שאנחנו בן שמאלי שלו. אם מגיעים לכלום סימן שאנחנו הכי גדולים (הכי ימניים).
 - קודם - מקביל לעוקב
 - הכנסה - ימינה/שמאלה עד שמגיעים ל- $null$ ואז הוספה.
 - מחיקה - אם עלה אז לאפס את הבן של ההורה, אם יש בן אחד אז לחבר הורה עם בן, אם יש שני בנים אז למצוא עוקב (שבטוח אין לו בן שמאלי כי אנחנו היחידים שקטנים ממנו) ואז להחליף את הערך שלו איתי ולהסיר אותו.
- נתונה סדרת מספרים. האם היא יכולה להיות סדרת מספרים חוקית בעת חיפוש? - צריך לצייר את העץ ולראות שלכל צומת התת עץ השמאלי קטן ממנו והתת עץ הימני גדול ממנו.
- 12.2-4: לא בהכרח כל הצמתים משמאל למסלול בעץ קטנים או שווים לו - ייתכן שמתישו לדוגמה יש פיצול שמאלה ואז יהיה צומת שגדול מהשורש.
- 12.2-5: אם לצומת יש שני בנים אז לעוקב אין בן שמאלי ולקודם אין בן ימני - נכון אחרת הבנים האלה היו קטנים מהעוקב או גדולים מהקודם
- 12.2-9: אבא של עלה הוא הכי קטן/גדול וקרוב אליו: כי קריאה עבור קודם/עוקב על הבן תחזיר את האבא.
- 12.3-3: מקרה הכי גרוע של מיון עץ חיפוש הוא שהסדרה כבר ממויינת (n^2), הכי טוב שהעץ מאוזן ($\lg n$)
- 12.3-4: מחיקה לא חילופית - לדוגמה העץ $214NN3N$.
- עצים שנבנים רנדומלית
 - משפט 12.4 - תוחלת הגובה של עץ שנבנה רנדומלית היא $O(\lg n)$.
 - עצי בסיס - כל צומת היא ערך, כל קשת היא הוספת תו. אפשר למיין לקיקוגרפית על ידי הכנסה לעץ והדפסת כל איבר בו.

פרק 13 - עצים אדומים-שחורים

- דרך לשמור על העץ מאוזן, ובכך כל הפעולות הן $O(\lg n)$
- תכונות - הורה, בן ימני, בן שמאלי, מפתח וצבע.
- תכונות האדום-שחור - כל צומת הוא אחד, שורש שחור, עלים שחורים, לכל צומת אדום יש בנים שחורים, כל המסלולים הפשוטים משורש אל עלה מכילים אותו מספר של צמתים שחורים.
- סיבובים
- הכנסה
- מחיקה

פרק 14 - הרחבה של מבני נתונים

- עץ ערכי מיקום - עץ אדום שחור שכל צומת מכיל את גודל העץ
 - מציאת ערך מיקום - מציאת הצומת ומעבר אל ההורה, בכל פעם שאנחנו בן ימני אנחנו מוסיפים את גודל הבן השמאלי.
 - מציאת איבר עם דירוג מסוים - מעבר ימינה/שמאלה בהתאם לדירוג לפי גודל עץ שמאלי.
 - יש צורך לתחזק את ערך הגודל בעת הכנסה והוצאה
- הרחבת מבנה נתונים - בחירת מבנה תשתית, קביעת מידע לתחזוק, אימות יכולת לתחזוק, פיתוח פעולות חדשות.
- משפט 14.1 הרחבת עץ אדום-שחור - כל שדה שאת תוכנו ניתן להסיק על פיו ועל פי בניו הישירים אז ניתן לתחזק את השדה במהלך הכנסה או מחיקה בלי להשפיע על זמן הריצה של הפעולות.

רקע מתמטי

$$\sum_0^n \frac{1}{k} = \ln n + O(1) \quad - \quad 7. \aleph \quad \bullet$$

$$\sum_0^\infty kx^k = \frac{x}{(1-x)^2} \quad - \quad 8. \aleph \quad \bullet$$

גידול של פונקציות - הוכחות

$$1 = \Theta\left(n^{\frac{1}{\lg n}}\right) : \text{מתקיים } n^{\frac{1}{\lg n}} = n^{\frac{\lg 2}{\lg n}} = n^{\log_n 2} = 2 \text{ וכמו כן הקשר מתקיים.}$$

$$1 = \Omega(\lg \lg^* n) : \text{אם נבחר } c_0 = 1 \text{ אז הטענה מתקיימת באופן טריוויאלי.}$$

$$\lg \lg^* n = \Omega(\lg^* n) : \text{נסמן } t = \lg^* n \text{ ונקבל } t \leq c \lg t. \text{ מתקיים } \lim_{x \rightarrow \infty} \frac{x}{\lg x} = \infty \text{ ולכן הטענה נכונה.}$$

$$\lg^* n = \Theta(\lg^* \lg n) : \text{ההפרש בערך מוחלט בין } \lg^* \lg n \text{ ל-} \lg^* n \text{ הוא אחד ולכן ניתן למצוא קבועים המקיימים את הקשר } \Theta().$$

$$\lg^* n = \Omega(2^{\lg^* n}) : \text{נסמן } t = \lg^* n \text{ ונקבל } t \leq c_0 t \leq 2^t, \text{ ועבור } c_0 = 1 \text{ זו טענה נכונה.}$$

$$\ln \ln n = \Omega(\sqrt{\lg n}) : \text{אם נסמן } t = \ln n \text{ ונבצע מעבר בסיסים נקבל את הביטוי } f(x) = \frac{1}{\sqrt{\ln 2}} \sqrt{t} - c \ln t \geq 0. \text{ עבור } 4 \leq n_0, c_0 = 1. \text{ פונקציה זו חיובית (ניתן להוכיח על ידי גזירה). ולכן הטענה נכונה.}$$

$$\sqrt{\lg n} = \Omega(\ln n) : \text{שקול לביטוי } \sqrt{\lg n} \left(\sqrt{\ln n} - \frac{c_0}{\sqrt{\ln 2}} \right) \geq 0 \text{ והוא אמיתי עבור } 4 \leq n_0, c_0 = 1.$$

$$\ln n = \Omega(\lg^2 n) : \text{שקול לביטוי } \ln n \left(\frac{1}{\ln^2 n} \ln n - c_0 \right) \geq 0 \text{ והוא אמיתי עבור } 2 \leq n_0, c_0 = 1.$$

$$\lg^2 n = \Omega(\lg^2 n) : \text{נסמן } m = \lg n \text{ ונקבל } \lg^2 n = \frac{\ln^2 n}{\ln^2 2} : \ln n = \Omega(\lg^2 n) \text{ כמו הקשר בין } n \text{ ו-} n^2.$$

$$\lg^2 n = \Omega(2^{\sqrt{2 \lg n}}) : \text{נסמן } m = \lg n \text{ ונקבל } m^4 \leq 2^{\sqrt{2} m} \text{ ואמנם } \lim_{x \rightarrow \infty} \frac{a^x}{x^k} = \infty \text{ ולכן הטענה נכונה.}$$

$$2^{\sqrt{2 \lg n}} = \Omega(2^{\lg n}) : \text{קשר זה ניתן להביע בצורה } c_0 \sqrt{2}^{\sqrt{8 \lg n}} \leq \sqrt{2}^{\lg n} \text{ כלומר } c_0 \leq \sqrt{2}^{\lg n - \sqrt{8 \lg n}} \text{ וזה מתקיים עבור } c_0 = 1, n_0 = 2.$$

$$\sqrt{2}^{\lg n} = \sqrt{n} : \text{קשר זה טריוויאלי.}$$

$$2^{\lg n} = \Theta(n) : \text{קשר טריוויאלי.}$$

$$n = \Omega(n \lg n) : \text{מתקיים עבור } c_0 = 1, n_0 = 2.$$

חשוב:

$$n \lg n = \Theta(\lg n!) : \text{הקשר שצריך להוכיח הוא היחס בין } n!, n^n$$

$$* \text{ כיוון ראשון: כל איבר בעצרת קטן או שווה ל-} n \text{ ולכן } n! \leq n^n$$

$$* \text{ כיוון שני: לפחות חצי מהאיברים גדולים או שווים ל-} \frac{1}{2}n \text{ ולכן } \left(\frac{1}{2}n\right)^{\frac{1}{2}n} \leq n! \text{ , לכן } \frac{n}{2} \lg \frac{n}{2} \leq \lg(n!) \leq \frac{1}{2}n \lg n - \frac{n}{2} = \frac{1}{2}n \lg n$$

$$n \lg n = \Omega(n^2) : \text{מתקיים עבור כל } c_0 = 1, n_0 = 2.$$

$$n^2 = \Theta(4^{\lg n}) : \text{שתי הפונקציות שוות ולכן הקשר טריוויאלי.}$$

$$n^2 = \Omega(n^3) : \text{נכון לפי שאלה 1.}$$

$$n^3 = \Omega(n^{\lg \lg n}) : \text{הביטוי בחזקה הימנית הוא פונקציה עולה ולכן קיימים אינסוף ערכי } n \text{ עבורם הביטוי הימני גדול מהביטוי השמאלי.}$$

$$n^{\lg \lg n} = \Theta(\lg n^{\lg n}) : \text{אם נסמן } n = 2^m \text{ נקבל בשני הביטויים } m^m \text{ ולכן הקשר מתקיים.}$$

$$2^n = \Omega(n 2^n) : \left(\frac{3}{2}\right)^n = \Omega(2^n) : \text{נכון עבור כל קבוע וכל } n \geq 1.$$

$$n 2^n = \Omega(e^n) : \text{בגלל החוק } \lim_{n \rightarrow \infty} \frac{a^n}{n^k} = \infty \text{ ניתן למצוא קבוע המקיים את הטענה.}$$

$$e^n = \Omega(n!) : \text{מתקיים לדוגמה עבור } c_0 = 1, n_0 = 10.$$

• $n! = \Omega((n+1)!)$: קשר זה הינו טריוויאלי עבור כל n טבעי.

• $(n+1)! = \Omega(2^{2^n})$: מתקיים $\frac{(n+2)!}{2^{2^{n+1}}} = \frac{n+2}{2^{2^{n+1}-2^n}} = \frac{n+2}{2^{2^n}} \rightarrow 0$ ולכן גם $\frac{(n+1)!}{2^{2^n}}$ שואפת ל-0 וקיים c_0 עבורו אי השיויון נכון.

• $2^{2^n} = \Omega(2^{2^{n+1}})$: הקשר הוא $c_0 2^{2^n} \leq 2^{2^{n+1}}$ כלומר $c_0 \leq 2^{2^{n+1}-2^n} = 2^{2^n}$ וזה מתקיים עבור $c_0 = 1, 2 \leq n_0$

(*)

• $\Theta(\lg n^{\lg a}) = \Theta(a^{\lg \lg n})$: הוכחה :

$$(\lg n)^{\lg a} = b \Rightarrow \lg a = \log_{\lg n} b = \frac{\lg b}{\lg \lg n} \Rightarrow \lg a \cdot \lg \lg n = \lg b \Rightarrow 2^{\lg a \cdot \lg \lg n} = a \cdot 2^{\lg \lg n} = 2^{\lg b} = b \quad \xleftarrow{\text{from start}} (\lg n)^{\lg a}$$

(*)

נמצא את היחס האסימפטוטי בין $\lg(n!)$ לבין $(\lg \lg n)!$:

נוכיח ש- $(\lg \lg n)! \leq \lg(n!)$ לכל $n \geq 2$. נפעיל $\lg$ על שני הצדדים :

$$\lg((\lg \lg n)!) = \lg(m!) = \Theta(\lg m^m) = \Theta\left(\lg\left((\lg \lg n)^{\lg \lg n}\right)\right)$$

$$\lg(\lg(n!)) = \lg(\Theta(n \lg n)) = \Theta(\lg(n \lg n)) = \Theta(\lg n + \lg \lg n) = \Theta(\lg n)$$

כלומר צריך להוכיח את יחס סדר הגודל בין $\lg n$ לבין $\lg((\lg \lg n)^{\lg \lg n})$. נסיר את ה $\lg$ ונקבל את הפונקציות $\lg \lg n^{\lg \lg n}, n$. נסמן $n = 2^{2^r}$ ונקבל את הפונקציות $2^{2^r}, 2^{2^r}$. הפונקציה השניה היא מסדר גודל גבוה יותר, לכן נקבל ש- $(\lg \lg n)! = O(\lg(n!))$

דרך נוספת :

$$(\lg \lg n)!, \lg n! \rightarrow \lg((\lg \lg n)!), \lg \lg n! \rightarrow \lg \lg n \cdot \lg \lg \lg n, \lg n \rightarrow \lg(\lg \lg n \cdot \lg \lg \lg n), \lg \lg n \rightarrow \lg \lg \lg n \leq \lg \lg n$$

נוסחאות נסיגה - דוגמאות

(*)

$$T(n) = 5T\left(\frac{n}{5}\right) + \frac{n}{\lg n}$$

נשתמש בשיטת עץ הרקורסיה: כל רמה i בעץ מתפצלת ל-5 ולכן יש 5^i צמתים בכל רמה. הערך של כל צומת הוא $\frac{\frac{n}{5^i}}{\lg \frac{n}{5^i}}$. ברמה של העלים ערך כל עלה הוא 1 ולכן הרמה האחרונה היא $\log_5 n$. לכן הפונקציה היא

$$T(n) = \sum_{i=0}^{\log_5 n} 5^i \cdot \frac{\frac{n}{5^i}}{\lg \frac{n}{5^i}} = n \sum_{i=0}^{\lg \frac{n}{5}} \frac{1}{\lg n - \lg 5^i} = n \sum_{i=0}^{\lg \frac{n}{5}} \frac{1}{\lg n - i \lg 5} = \frac{n}{\lg 5} \sum_{i=0}^{\lg \frac{n}{5}} \frac{1}{\frac{\lg n}{\lg 5} - i} \stackrel{\text{היפוך סדר המחוברים}}{=} \frac{n}{\lg 5} \sum_{i=0}^{\lg \frac{n}{5}} \frac{1}{i} \stackrel{\text{משפט 7.4}}{=} \frac{n}{\lg 5} \lg \lg \frac{n}{5} = \Theta(n \lg \lg n)$$

(*)

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n$$

נשתמש בשיטת עץ הרקורסיה: כל צומץ בעץ הוא מהצורה $\frac{n}{2^a 4^b 8^c}$ כאשר $a + b + c = i$. הסכום של כל הצמתים באותה הרמה הוא

$$\sum_{a+b+c=i} \frac{n}{2^a 4^b 8^c} = n \sum_{a+b+c=i} \left(\frac{1}{2}\right)^a \left(\frac{1}{4}\right)^b \left(\frac{1}{8}\right)^c = \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8}\right)^i = \left(\frac{7}{8}\right)^i$$

והפונקציה היא הסכום של כל הרמות:

$$T(n) = \sum_{i=0}^{\lg n} n \left(\frac{7}{8}\right)^i = n \sum_{i=0}^{\lg n} \left(\frac{7}{8}\right)^i = n \cdot \frac{1}{1 - \frac{7}{8}} = 8n = \Theta(n)$$

תרגול למבחן

• 2024 69

– 4א החציון של איברי המסלול הימני - $1 - 2^{\lceil \frac{\lg n}{2} \rceil}$

– הכנסה לערימה לוקחת $\Omega(n \lg n)$ במקרה הגרוע - $\Omega(n \lg n) = \Theta(n \lg n) = \Theta(\lg n!) = \Theta(\sum \lg i)$

– מבנה שאפשר להכניס אליו ב- $O(n)$ ולמצוא טווח בין שני מספרים ב- $O(\lg n)$ - עץ שחור אדום שאת הבניה מבצעים כמו בערמה ואת הטווח על ידי הכנסת שני המפתחות, מציאת ה- $rank$ שלהם וחיסור, ואז הוצאתם.

• 2023 ב26

– 1 -

* בהוכחת מיון לשים לב להוכיח אינדוקציה חזקה - לא רק ממינים את n אלא כל מספר קטן ממנו.

* *select* אופטימלי למציאת חציון, *partition* אופטימלי לחלוקה סביב איבר, *mergesort* אופטימלי למיון.

– 2 - מימוש תור על ידי שתי מחסניות - דוחפים לראשונה ואם היא לא ריקה אז לשניה, בהוצאה מוציאים מהראשונה ואם הראשונה לא ריקה אז הופכים את השנייה על הראשונה. ככה תמיד בראשונה האיברים נמצאים בסדר ההכנסה.

– 3 - הוצאת k איברים קטנים בזמן $O(n + k \lg k)$ - מציאת איבר עם ערך מיקום k (בזמן n), ביצוע *partition* (בזמן n), מיון באמצעות *mergesort* (בזמן $k \lg k$).

– 4 - שחזור עץ חיפוש בינארי - לא ניתן לבצע מסריקה תוכית (קל למצוא דוגמאות נגדיות), לא ניתן לבצע מסריקה תחילית (אם יש שני מספרים זהים לא ידוע מי מעל מי), ניתן לבצע מסריקה תחילית אם כל האיברים שונים (בצורה רקורסיבית לוקחים איבר ומחפשים את האיבר הבא שגדול ממנו, כל מה שביניהם זה תת עץ שמאלי והשאר ימני).

– 5 - מבנה נתונים שמאפשר קבלת ממוצע ב- $O(\lg n)$ - הוספת שדה סכום, הכנסת הערכים a, b וחיסור הסכום וערך המיקום של אחד מהשני.

• 2023 ב99 ?

• 2023 א91 -

– 2 - אם מקבלים את מספר הקלטים האפשריים k אפשר להסיק שגובה עץ ההחלטות הוא לפחות $\lg(k)$ אבל מקסימום 2^h . מכך אפשר להסיק על ערך h וזהו הזמן במקרה הגרוע.

– 3 - אם יודעים מרחק מקסימלי בין שני איברים במערך אפשר למצוא מינימום ומקסימום ולבנות מערך מניה בגודל ההפרש, ומובטח שהמיון ירוץ ב- $\Theta(n)$.

– 4 - מספר הקריאות ל-*random* ב-*randomized quicksort* - בפועל זהה למספר הקריאות ל-*quicksort*. $T(n) = T(k) + T(n-k) + 1$, באמצעות שיטת הצבה אפשר להוכיח $T(n) = O(n)$. מספר הקריאות במקרה הטוב הוא n כי תמיד בסוף מגיעים לאיבר אחד בכל עלה.

• 2020 ב81

– 1 - לחלק בחזקה הגבוהה, הצבת משתנה $n = 2^m$. Θ לא אומר שהגבול הוא 1 - לדוגמה $n, 2n$.

– 2 - גם אם עושים מיון מהיר רק על טווח מסוים במערך אז המקרה הגרוע והטוב אותו דבר כי ייתכן שהם בטווח.

– 3 - אם רוצים לעדכן ערכים בערמה - צריך לעדכן בעת החלפת מספרים ב-*heapify* או ב-*insert*.

– 4 - הכנסה לעץ חיפוש

* זמן הריצה של בניית העץ - במקרה הגרוע גובה העץ גדל $(\sum \lg i = \lg n! = n \lg n)$, במקרה הטוב הוא נבנה כמו ערימה ואז $O(n)$.

* האם התשובה תשתנה אם הייתה מחיקה - כן כי ייתכן שהענף קטן ב-1.

* האם התשובה תשתנה אם הייתה הכנסה לעץ שחור אדום - כן כי ייתכן שהיה סיבוב.

5 - מבנה שאפשר לבנות בזמן לינארי, למחוק מפתח חיובי קטן בזמן רגיל ולקבל שכח בזמן מיידי - לבנות עץ א"ש ממערך ממויין (לינארי), ערימת מינימום עבור מפתחות חיוביים וערימת מקסימום עבור שכיחויות, האיברים בערימות מכילים מצביעים לעץ וההפך (אפשר לעשות על ידי חיפוש בעץ לאחר הכנסה).

• 2022ג 82 -

1 - אם $\lg f(n) = O(\lg g(n))$ או $f = O(g)$ - לא נכון, לדוגמה $f(n) = n, g(n) = n^2$

2 - החלפת ראשון ואחרון וביצוע חלוקה לא ממיין כי ייתכן שמכניסים איבר לא קשור לתחום הממויין.

3 -

* א - השוואות שמבוצעות על מיון ערמה של מערך ממויין הפוך שהוא עץ שלם - כל איבר יורד עד הסוף ($2h$ השוואות) לכן התשובה היא סכום כל הגבהים. סכום כל הגבהים בעץ שלם שגובהו k הוא $2^{h+1} - h - 2$ (הוכחה באינדוקציה).

* ב - רוצים להשוות 1000 מספרים בני 4 בתים. האם ערימה או בסיס יותר יעיל?

· ערמה - זמן $50000 \approx 4000 \cdot 10 + 2 \cdot 4000 \lg 4n$, בשביל מקום צריך ליצור מערך באורך $n = 1000$.

· בסיס - זמן $10000 \approx 4(2 \cdot 1000 + 2 \cdot 256) = d(2n + 2k)$, בשביל מקום צריך $n + k = 1256$.

4 -

* הדפסת איבר מינימלי מסריקה תחילית - מחפשים פעם ראשונה שהאיבר הבא גדול יותר או סוף הרשימה (כי בהתחלה הולכים הכי שמאלה).

* הדפסת כל האיברים שלפניהם יש רק איברים קטנים - עבור כל איבר אם האיבר שבראש המחשנית קטן ממנו אז מוציאים, בסוף אם המחשנית ריקה סימן שזה באמת ככה, בסוף מכניסים את האיבר למחשנית (המחשנית תכיל מקסימום קודם כלשהו ואז רצף של איברים שלא גדולים מהמקסימום, ככה עד המקסימום הבא או סוף הרשימה).

5 -

* הדפסת הממוצע של כל האיברים שקטנים מאיבר מסוים ונמצאים שמאלה ממנו - הכנסה לעץ א"ש שמכיל גם סכום ועבור כל אחד הכנסה לעץ, מציאה וחישוב ממוצע כמו $os - rank$.

* הוספת שורה שמחזירה הפרש מינימלי שקטן מערך מסוים, בין שני איברים - שמירה עבור כל צומת מינימום, מקסימום ואת שני האיברים, אם הערך הוא בתת-עץ השמאלי אז הפעלה רקורסיבית עבורו, אם הוא בתת עץ הימני אז לבדוק אם הוא ביני לבין הבן הימני, בגדול להשוות בין 4 ערכים - הפרש בתת עץ שמאלי, הפרש בין השורש למקסימום של בן שמאלי, הפרש בין שורש למינימום של בן ימני, הפרש של תת עץ ימני.

• ממנים

- ממן 12

* מספר ההשוואות המבוצעות על בניית ערמה עם n איברים זהים - $n - 1$ (2) השוואות עבור כל הורה, יש קצת הבדל האם יש בן אחרון או לא)

* חישוב אורך ערמה - מעבר על בנים שמאליים ואז חיפוש בינארי על רמה אחרונה.

* מקרים טובים וגרועים:

· הכנסה - מטוב ממויין (n) , מקרה גרוע ממויין הפוך (n^2)

· מיזוג - בטוב המערכים מתפצלים ככה שכל הקטנים נמצאים בפיצול הקטן (ואז הגדול נלקח כמו שהוא), בגרוע האיברים מתפצלים בצורה שזורה ככה שאף פעם לא לוקחים זנב אלא משווים את כל האיברים (בשניהם $n \lg n$)

· ערמה - בטוב אף מספר לא מקודם עם $heapify$, רק עוברים על כל האבות $(\frac{n}{2})$, בגרוע המערך ממויין הפוך וכל מספר יורד עד הסוף - $n \lg n$.

• מהיר - בטוב המערך מחולק ל-2 כל פעם $(n \lg n)$, ברע מחלקים ל- $(0, n-1)$ בזמן n^2 .

* לא ניתן למיין ערמת מינימום בזמן לינארי - אחרת היה אפשר למיין על ידי הכנסה ואז מיון הערמה

* ניתן למיין $\frac{n}{\lg n}$ איברים קטנים בזמן לינארי - הכנסה והוצאה מערמת מינימום.

* ניתן למיין מערך בגודל n שכל מספר בו הוא לכל היותר $2n\sqrt{n}$ - נכון כי מפצלים לעד 4 (זמן קבוע) וגדול מ-4 (נכנס ב- $n^2 - 1$) ואת זה אפשר למיין לפי מנייה).

* מציאת מספר חסר במערך בזמן לינארי - חיסור סכום תיאורטי מהסכום האמיתי.

– ממך 14

* הכנסת "סימניות" לתוך מערך כדי להגדיל חיפוש

• ברמה אחת - להכניס $\sqrt{n}$ סימניות כל $\sqrt{n}$ איברים.

• בשתי רמות - רמה ראשונה - להכניס $n^{\frac{1}{3}}$ סימניות, כל $n^{\frac{2}{3}}$ איברים, רמה שנייה - להכניס $n^{\frac{1}{3}}$, כל $n^{\frac{1}{3}}$ איברים (שהם שורש של $n^{\frac{2}{3}}$).

• ניתן להכליל ל- n רמות

* נניח שקיים אלגוריתם להדפסת כמות איברים שקטנים מהמערך ונמצאים משמאלו

• חייב להיות $n \lg n$ כי אחרת אפשר למיין על ידי להפוך ולהכניס למערך לפי מיקום

• אלגוריתם - הכנסת לעץ ערכי מיקום לפי הסדר ומציאת ערך המיקום מיידית.

* 4 - עץ א"ש שתומך בהיפוך סימן, חיפוש אי שלילי שגדול או שווה מסוים וחציון שליליים -

• היפוך סימן על ידי הכנסה והוצאה

• שימוש בעץ כדי לאחסן חיוביים (עם מצביעים דו כיווניים) וחיפוש אי שלילי מינימלי על ידי חיפוש האיבר ואם הוא לא קיים

אז הכנסתו ואז ביצוע *successor*.

• חציון שליליים על ידי הוספת *size* שסופר שליליים.

• 2022 א 87

– 22 - כדי לעשות *heap – increase – key* ב- $O(\lg h)$ במקום $O(\lg n)$ אפשר להעתיק הצידה את האיברים במסלול מהצומת אל הראש ואז לעשות חיפוש בינארי.

– 3 - ערך מופיע $\frac{n}{3}$ ושני מופיע $\frac{n}{4}$ - אחד בודקים ב- $\frac{2n}{3}, \frac{n}{3}$ ושני ב- $\frac{3n}{4}, \frac{2n}{4}, \frac{n}{4}$.

– 4 - מיוזג k עצי חיפוש בינאריים עם סהכ n איברים ב- $O(n \log k)$ - הדפסה תוכית של כל עץ, הכנסת ראשי כל רשימה לערימת מינימום שכולל את מספר הערמה, כאשר מוציאים איבר אז מכניסים את הבא מאותה הרשימה עד שמגיעים לסוף. גובה ערימת המינימומים היא $\log k$ ומכאן הסיבוכיות

• הוכחת נכונות עץ חיפוש בינארי - יש n עלים, ומקסימום 2^h עלים ולכן $\lg n \leq h$.

• חיפוש איבר בעץ שקרוב לערך נתון מלמעלה ב- $\lg n$ - חיפוש בעץ, אם לא קיים אז להכניס, למצוא עוקב ולהוציא.

• איך אפשר לבנות עץ אדום שחור בזמן לינארי עם מערך ממויין - באופן רקורסיבי מגדירים שהשורש הוא האמצע, חצי שמאלי לתת עץ שמאלי וחצי ימני לתת עץ ימני. (אפשר להוכיח באינדוקציה שלמה - מפצלים לשני חלקים, כל אחד מאוזן לפי ההנחה וגם הכלל מאוזן).

$$\lg(2 \cdot \dots \cdot n) = \lg(2^n \cdot 1 \cdot \dots \cdot \frac{n}{2}) = \lg 2^n + \lg(\frac{n!}{2}) = n + \Theta(\frac{n}{2} \lg \frac{n}{2}) = \Theta(\frac{n}{2} \lg \frac{n}{2}) = \Theta(n \lg n)$$

$$(4 \leq n) \quad \frac{n}{4} \leq \frac{n}{2} - 1 \leq \lfloor \frac{n}{2} \rfloor \leq \frac{n}{2}. \quad (6 \leq n) \quad \lg \frac{n}{2} \leq \lg \lceil \frac{n}{2} \rceil \leq \lg(\frac{n}{2} + 1) \leq \lg \frac{2n}{3}$$

• מותר לצטט למבחן ממדריך הלמידה: שיטת האב המורחבת, פונקציות שכיחות בשיטת האב, שגרת מחיקת איבר מערימה, שיטת עכרה באזי.

• מועד ??

– 1 - קביעת סדר גודל בין $\lg(n!)$ לבין $(\lg \lg n)!$: $\lg n! = n^n$ - קיים בחלק "סדרי גודל"

• מועד ??

– 4 - צריך ליצור אלגוריתם שמקבל ערימת מינימום בת n איברים ומספר k , מחזיר את k האיברים הקטנים ביותר

* ב- k^2 - לוקחים $1 - 2^{k+1}$ מהערימה (k הקטנים חייבים להיות שם כי זה גובה הערימה) וממיינים אותם - $k \cdot \lg h = k^2$.

* ב- $k \lg k$ - עוברים על הערימה מבלי לשנות אותה, מכניסים "מועמדים" אל הערימה ומוציאים את המינימום. גובה הערימה הוא לא יותר מ- $k \lg k$ ולכן התהליך הוא $k \lg k$

• דוגמה נוספת - אם נתון שבמערך כל איבר נמצא במרחק k ביחס למיקום שלו אפשר להכניס אותם לערימה ולהוציא את המינימום, חסכוני ביחס למיון רגיל.

• הערות לגבי הוכחת עץ רקורסיה -

– כדאי לבדוק את הסכומים של כמה רמות ואז אפשר להניח מה הסכום בכל רמה ולעשות את החישוב, אם ממש רוצים להוכיח פורמלית אפשר אחר כך לעשות שיטת ההצבה

– אם יש סכום מ-0 עד $\lg n$ אפשר לחסום אותו בסכום עד אינסוף.

• 2022 א 89

– 4 - כדי למיין מערך בעל $\lg n$ איברים שונים בזמן $n \lg \lg n$ - להכניס לעץ א"ש כאשר כל צומת סופר כמה כפילויות יש ואז להדפיס

– 5 - כדי להוסיף ערך לתת עץ שלם בזמן $O(\lg n)$ אפשר להוסיף שדה לצומת שאומר כמה צריך להוסיף לערכים שתחתיו בעת *insert*

• 2022 ב 82

– 1 - יש מצב שפרמטר יגרום לאיפוס של $f(n)$!

– 4 -

* כל איבר בתת עץ קטן משורש (לא רק בן ישיר)

• 2022 ב 85

– 1 - $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n}$ - עץ רקורסיה,

• 2022 ב 95

– 1 - $\sum i \lg i \leq \int x \lg x \approx \int x \ln x = \frac{1}{2} (x^2 \ln x - x)$

• נוסחאות נסיגה - אפשר לחלק בחזקה הגבוהה